

整数論 Euclid の互除法 Bézout の等式

Euclid の互除法

解説動画はこちらです。

1. 概要

本講座では、**Euclid の互除法** (Euclidean Algorithm) について解説します。

Euclid の互除法は 2 つの整数の最大公約数 (gcd) を求めるアルゴリズムとして有名です。しかし、その応用は最大公約数を求めることだけにとどまりません。

特に Euclid の互除法の応用として導かれる **Bézout の等式**は重要で、そこから次のような初等整数論の結果（あるいはそれを求めるアルゴリズム）が次々と導かれます。

- mod 乗算に関する逆元の存在条件, 計算方法.
- 1 次合同式 $ax \equiv b \pmod{m}$ の解法.
- 素因数分解の一意性.
- 中国剰余定理.

本講座ではこのうち上の 2 項目を解説します。

2. 前提知識

AtCoder Algorithm Lectures における次の講座の内容を前提とします。

- 合同式の基礎

3. 最大公約数

【定義 1】

整数 a, b に対し、その**最大公約数** (greatest common divisor, gcd) $\gcd(a, b)$ を、次のように定義する。

- $a = b = 0$ の場合 : $\gcd(a, b) = 0$.
- a, b のうち少なくとも一方が 0 でない場合 : a, b 両方の約数であるような正整数 (公約数) のうち最大のもの。

ここで後者の場合について、1 が a, b の公約数であることと、 $a \neq 0$ ならば公約数は $|a|$ 以下であること、 $b \neq 0$ ならば公約数は $|b|$ 以下であることから、「最大のもの」の存在が保証される。

▶ 注意

例えば次が成り立ちます。

- $\gcd(30, 40) = 10$.
- $\gcd(-30, 40) = 10$.
- $\gcd(-30, -40) = 10$.
- $\gcd(100, 50) = 50$.
- $\gcd(23, 32) = 1$.
- $\gcd(0, 123) = 123$.
- $\gcd(0, -123) = 123$.
- $\gcd(0, 0) = 0$.

次節で用いられる gcd の性質を確認しておきます。

【命題 2】

a, b, k を整数とすると $\gcd(a - kb, b) = \gcd(a, b)$ が成り立つ。

$b = 0$ の場合は明らかなので、そうでないとします。両辺は公約数の最大値ですが、「公約数」が両辺で一致することを示せばよいです。したがって、

- d が $a - kb, b$ の公約数ならば、 a, b の公約数でもあること。
- d が a, b の公約数ならば、 $a - kb, b$ の公約数でもあること。

を示せばよいです。前者は、 $a = (a - kb) + kb$ であることと d の倍数の和が d の倍数であることから分かります。後者も同様です。 ■

4. Euclid の互除法

本節では、 $\gcd(a, b)$ を $O(\log \min(|a|, |b|))$ 時間で求めるアルゴリズムである Euclid の互除法 (Euclidean Algorithm) について解説します。

$\gcd(a, b) = \gcd(|a|, |b|)$ なので、本節では a, b は**非負整数であることを仮定**します。

4.1. アルゴリズムと具体例

【命題 2】 より $b > 0$ のとき、 $\gcd(a, b) = \gcd(a \bmod b, b)$ が成り立ちます。同様に $a > 0$ のとき $\gcd(a, b) = \gcd(a, b \bmod a)$ も成り立ちます。したがって、 a, b のうち大きい方 ($a = b$ の場合はどちらでもよい) をもう一方で割った余りに置き換えても $\gcd$ は変わりません。

このことを反復すると a, b を小さくしていくことができ、最終的に a または b を 0 にできて、 a, b の最大公約数が求まります。このアルゴリズムを **Euclid の互除法 (Euclidean Algorithm)** といいます。

次の例は、Euclid の互除法を用いて $\gcd(15, 39)$ を計算する例です。

- $39 \bmod 15 = 9$ である。よって $\gcd(15, 39) = \gcd(15, 9)$ である。
- $15 \bmod 9 = 6$ である。よって $\gcd(15, 9) = \gcd(6, 9)$ である。
- $9 \bmod 6 = 3$ である。よって $\gcd(6, 9) = \gcd(6, 3)$ である。
- $6 \bmod 3 = 0$ である。よって $\gcd(6, 3) = \gcd(0, 3)$ である。
- 明らかに $\gcd(0, 3) = 3$ である。よって以上のことと合わせて $\gcd(15, 39) = 3$ と計算できる。

4.2. Euclid の互除法の実装

4.1 節では説明のため、「 a, b のうち大きい方を小さい方で割った余りに置き換える」という説明をしました。実装では、毎回大小比較を行うのではなく、アルゴリズムの進行中で（最初の例外を除き）常に $a > b$ が成り立つようにするものがよく用いられます。

よく知られている 2 通りの実装方法を紹介します。これらは実装の見た目は異なりますが、行われる計算は全く同じです。

```
def gcd(a, b):
    while b != 0:
        a = a % b
        a, b = b, a
    return a
```

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

アルゴリズムの進行は 4.1 節で説明したものとは厳密には一致せず、最初に 1 回だけ小さい方を大きい方で割る（値は変化しない）操作が行われる可能性もあります。

4.3. 計算量解析

本節では Euclid の互除法の計算量を、必要な除算回数という形で評価します。

なお、4.1 節で扱ったように、除算といえば常に大きい方をもう一方で割るという形の除算のみを数えることとします（4.2 節の実装で最初に行われる可能性がある値が変化しない除算は数えません）。

$a \geq b > 0$ であるとき、 $(a \bmod b) \leq \frac{1}{2}a$ が成り立つので、Euclid の互除法において 2 回の除算を行うごとに a, b それぞれが $\frac{1}{2}$ 倍以下に減少します。よって $O(\log \min(a, b))$ 回の除算によって一方が 0 である状態に到達します。

▶ $(a \bmod b) \leq \frac{1}{2}a$ が成り立つ理由

したがって次の定理が成り立ちます.

【定理 3】

Euclid の互除法による $\gcd(a, b)$ の計算における除算の回数は $O(\log \min(a, b))$ 回である.

$\min(a, b)$ の大きさに対してどれだけの除算回数が必要となるかについて, より詳しく最悪ケースを見積もることもできます. 予想はしやすいことだと思いますが, この最悪ケースを与えるのは常に商が 1 になる場合で, **Fibonacci 数列** の場合に達成されます. Fibonacci 数列 F_n を次の初期値と漸化式によって定義します.

$$\begin{aligned} F_0 &= 0, & F_1 &= 1, \\ F_n &= F_{n-1} + F_{n-2} & (2 \leq n) \end{aligned}$$

【定理 4】

非負整数 a, b が $a \geq b$ かつ $a \neq 0$ を満たすとする. Euclid の互除法により $\gcd(a, b)$ を計算する際の除算回数が n 回であるとき, $a \geq F_{n+1}$ かつ $b \geq F_n$ が成り立つ.

n に関する帰納法により証明します. $n = 0$ のときは明らかです.

n に対する主張が正しいと仮定します. $a \geq b$ かつ $a \neq 0$ に対して Euclid の互除法の除算回数が $n + 1$ であるとし, このとき帰納法の仮定より, $b \geq F_{n+1}$ かつ $(a \bmod b) \geq F_n$ が成り立ちます. $a \geq b$ より $a \geq b + (a \bmod b) \geq F_{n+1} + F_n = F_{n+2}$ が成り立つので, $a \geq F_{n+2}$ かつ $b \geq F_{n+1}$ が成り立ちます. よって帰納法により定理が示されました. ■

【系 5】

Euclid の互除法による $\gcd(a, b)$ の計算における除算の回数は, $F_n \leq \min(a, b)$ となる最大の n 以下である.

なお, Fibonacci 数列 F_n はおよそ $\frac{1}{\sqrt{5}} \cdot \left(\frac{1 + \sqrt{5}}{2} \right)^n$ 程度であることが知られており, このことを利用して **【系 5】** を書き換えることも可能です. 例えば除算回数がより手軽に評

価できる定理として、除算回数は $\min(a, b)$ の桁数の 5 倍以下であることが知られています (Lamé の定理)。

5. 拡張 Euclid の互除法と Bézout の等式

5.1. Bézout の等式

【定理 6】(Bézout の等式)

任意の整数 a, b に対して、

$$ax + by = \gcd(a, b)$$

を満たす整数 x, y が存在する。この x, y は Euclid の互除法と同様の $O(\log(\min(|a|, |b|)))$ 時間で計算できる。

拡張 Euclid の互除法と呼ばれるアルゴリズムによりこの x, y を計算する方法を解説します。これは、Euclid の互除法で $\gcd(a, b)$ を計算する際に現れる数をすべて、 $ax + by$ の形で表すものです。4.1 節と同じ具体例で確認してみましょう。 $a = 15, b = 39$ とします。

- $15 = a (= 1a + 0b), 39 = b (= 0a + 1b)$ である。
- $39 \bmod 15 = 9$ である。商は 2 であるから、 $9 = 39 - 2 \cdot 15 = b - 2a = -2a + b$ である。
- $15 \bmod 9 = 6$ である。商は 1 であるから、 $6 = 15 - 1 \cdot 9 = a - (-2a + b) = 3a - b$ である。
- $9 \bmod 6 = 3$ である。商は 1 であるから、 $3 = (-2a + b) - (3a - b) = -5a + 2b$ である。
- $6 \bmod 3 = 0$ である。よって $\gcd(a, b) = 3 = -5a + 2b$ であると分かるので、この場合には $x = -5, y = 2$ が **【定理 6】** を満たす。

5.2. 実装例

Euclid の互除法で小さくしていく 2 数を s, t として、 x_s, y_s, x_t, y_t を $s = ax_s + by_s, t = ax_t + by_t$ を満たす整数とすると、例えば次のような実装が可能です。

```
# return (x, y) such that ax + by = gcd(a, b)
def extgcd(a, b):
    s = a
    t = b
    xs, ys = 1, 0
    xt, yt = 0, 1
    while t:
        u = s // t
        s -= t * u
        xs -= xt * u
        ys -= yt * u
        s, t = t, s
        xs, ys, xt, yt = xt, yt, xs, ys
    return xs, ys
```

AtCoder Library では、このうち a の係数である x_s, x_t のみを保持するという実装を用いています。

- 実装例：[AtCoder Library, internal_math, `inv\\_gcd`](#)

▶ 参考：別のアルゴリズム

6. m を法とする乗法逆元

【定義 7】

m を正整数、 a を整数とする。 $ax \equiv 1 \pmod{m}$ を満たす整数 x を、 m を法とする a の **乗法逆元** (modular inverse) という。

m を法として考えていることが文脈上明らかな場合には、単に「乗法逆元」ともいいます。あるいは「mod 逆元」あるいは単に「逆元」などと呼ばれることもあります。（ただし「逆元」はかなり広く用いられる用語なので、合同式についての話であることが明確な文脈以外では誤解を招く可能性もあります。）

例をいくつか挙げておきます。

- 3 の 10 を法とする乗法逆元として、7, 17, -3 などがある。

- 4 の 10 を法とする乗法逆元は存在しない.
- 0 の 1 を法とする乗法逆元として, $0, 1, 2$ などがある.

乗法逆元は存在しないこともあります. また存在する場合には, それらはすべて m を法として合同であることが以下で示されます.

【定義 8】(互いに素)

整数 a, b は $\gcd(a, b) = 1$ であるとき, **互いに素** (coprime) であるという.

【定理 9】

m を正整数, a を整数とする. a の $\text{mod } m$ での乗法逆元が存在することは, a と m が互いに素であることと同値である.

$g = \gcd(a, m)$ とします.

まず a の乗法逆元 x が存在するとします. このとき $ax - 1$ は m の倍数なので g の倍数です. また a は g の倍数なので ax も g の倍数です. したがって g はこれらの差である 1 の約数なので, $g = 1$ となります. つまり a と m は互いに素です.

逆に a と m が互いに素であるとします. このとき Bézout の等式より, $ax + my = 1$ を満たす整数 x, y が存在します. この x が a の $\text{mod } m$ での乗法逆元となります.

以上で定理が証明されました. ■

a に乗法逆元があるとき, 次の命題のように「 a による割り算」に相当する計算が行えます.

【命題 10】

a が $\text{mod } m$ での乗法逆元を持つとし, そのひとつを a_{inv} とする. このとき, 任意の整数 b に対して次が成り立つ:

$$ax \equiv b \pmod{m} \iff x \equiv a_{\text{inv}} \cdot b \pmod{m}.$$

$\implies$ は, 次のようにして示すことができます.

$$x \equiv 1 \cdot x \equiv a_{\text{inv}} a \cdot x \equiv a_{\text{inv}} \cdot ax \equiv a_{\text{inv}} \cdot b \pmod{m}.$$

$\impliedby$ は, 次のようにして示すことができます.

$$ax \equiv a \cdot a_{\text{inv}} \cdot b \equiv aa_{\text{inv}} \cdot b \equiv 1 \cdot b \equiv b \pmod{m}.$$

以上で示されました. ■

特に $b = 1$ とすることで, 乗法逆元は m を法として一意であることが分かります.

【問題 11】

1. 14 と 33 は互いに素です. 14 の 33 を法とする乗法逆元をひとつ求めてください.
2. $14x \equiv 25 \pmod{33}$ を満たす x であって $0 \leq x < 33$ を満たすものを求めてください.

▶ 解説

より一般に x を未知数とする合同式 $ax \equiv b \pmod{m}$ の解について整理すると次のようになります.

【命題 12】

m を正整数, a, b を整数とし, 合同式 $ax \equiv b \pmod{m}$ を満たす整数 x について考える. $g = \gcd(a, m)$ とする.

b が g の倍数でないならば, $ax \equiv b \pmod{m}$ を満たす整数 x は存在しない.

b が g の倍数ならば, $ax \equiv b \pmod{m}$ を満たす整数 x は m/g を法として唯一存在する. つまり $ax \equiv b \pmod{m} \iff x \equiv x_0 \pmod{m/g}$ を満たす整数 x_0 が存在する.

$ax \equiv b \pmod{m}$ が成り立つとき, ax, m はともに g の倍数であることから b も g の倍数となります. したがって b が g の倍数でないならば $ax \equiv b \pmod{m}$ を満たす整数 x は存在しません.

次に b が g の倍数であるとします. このとき a, b, m はすべて g の倍数なので, $a = ga', b = gb', m = gm'$ となる整数 a', b', m' が存在します. すると $\gcd(a', m') = 1$ であり,

$$ax \equiv b \pmod{m} \iff a'x \equiv b' \pmod{m'}$$

となるので、【命題 10】よりこれを満たす x は $m' = m/g$ を法として唯一存在します。 ■

【系 13】

a, b, c を整数とし、 $g = \gcd(a, b)$ が 0 でないとする。 c が g の倍数でないならば、このとき、 $ax + by = c$ を満たす整数 x, y は存在しない。

c が g の倍数ならば、ある整数 x_0, y_0 が存在して、整数 x, y について次の 2 つは同値である。

- $ax + by = c$ が成り立つ
- ある整数 t について $x = x_0 + (b/g)t, y = y_0 - (a/g)t$ が成り立つ。

例えば $b \neq 0$ であるとする、 $ax + by = c$ の解となる x 全体は、 $ax \equiv c \pmod{|b|}$ となる x 全体です。 c が g の倍数であるとき、【命題 12】より x は $|b|/g$ を法として一意に存在し、このことから証明できます。 ■

7. 公約数・公倍数

公約数、公倍数について成り立つことについて補足します。

【命題 14】

a, b を整数とする。 このとき d が a, b の公約数であることと、 d が $\gcd(a, b)$ の約数であることは同値である。

$\gcd(a, b)$ は a, b の公約数なので、 $\gcd(a, b)$ の約数が a, b の公約数であることは明らかです。

次に d が a, b の公約数であるとします。 Bézout の等式より $ax + by = \gcd(a, b)$ を満たす整数 x, y が存在します。 仮定より a, b は d の倍数なので、 $\gcd(a, b)$ は d の倍数です。 したがって d は $\gcd(a, b)$ の約数となります。 ■

【定義 15】

整数 a, b に対し, その**最小公倍数** (least common multiple, lcm) $\text{lcm}(a, b)$ を, 次のように定義する.

- $a = b = 0$ の場合: $\text{lcm}(a, b) = 0$.
- a, b のうち少なくとも一方が 0 でない場合: a, b 両方の倍数であるような正整数 (公倍数) のうち最小のもの.

【命題 16】

a, b を正整数とすると, 次の命題が成り立つ.

- $\text{lcm}(a, b) = \frac{ab}{\text{gcd}(a, b)}$.
- n が a, b の公倍数であることと, n が $\text{lcm}(a, b)$ の倍数であることと同値である.

a, b の公倍数がどのようなものかを考えましょう. このような数は, 整数 x によって ax と書ける数のうちで, b の倍数であるものです.

$g = \text{gcd}(a, b)$ とすると, $ax \equiv 0 \pmod{b}$ の解は, 【命題 12】より b/g の倍数全体です. よって最小公倍数は $a \cdot \frac{b}{g} = \frac{ab}{g}$ ですし, その他の公倍数はすべてその倍数となります. ■

8. 関連問題

1. https://atcoder.jp/contests/math-and-algorithm/tasks/math_and_algorithm_o
2. https://atcoder.jp/contests/typical90/tasks/typical90_v
3. https://atcoder.jp/contests/abc109/tasks/abc109_c
4. https://atcoder.jp/contests/arc051/tasks/arc051_b
5. https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E
6. https://atcoder.jp/contests/abc186/tasks/abc186_e
7. <https://codeforces.com/problemset/problem/530/C>
8. https://atcoder.jp/contests/abc340/tasks/abc340_f
9. https://atcoder.jp/contests/abc315/tasks/abc315_g

10. https://atcoder.jp/contests/typical90/tasks/typical90_al

11. https://atcoder.jp/contests/abc070/tasks/abc070_c

1, 2, 3, 4 は最大公約数の計算について問う問題です.

5, 6, 7, 8, 9 は Bézout の等式, $ax \equiv b \pmod{m}$ や $ax + by = c$ などの整数解についての理解を問う問題です.

10, 11 は最小公倍数に関する問題です.

9. まとめ

本講座では, Euclid の互除法, 拡張 Euclid の互除法について解説しました.

特に拡張 Euclid の互除法から導かれる Bézout の等式は重要で, 初等整数論の主要な定理やアルゴリズムの基礎となります. AtCoder Algorithm Lectures では以下の講座で学習を進めていただければと思います.

- [素数の性質](#)
- [中国剰余定理](#)

10. 参考文献

1. cp-algorithms. Euclidean algorithm for computing the greatest common divisor. <https://cp-algorithms.com/algebra/euclid-algorithm.html>. (閲覧日: 2026-03-31).
2. cp-algorithms. Extended Euclidean Algorithm. <https://cp-algorithms.com/algebra/extended-euclid-algorithm.html>. (閲覧日: 2026-03-31).
3. USACO Guide. Extended Euclidean Algorithm. <https://usaco.guide/adv/extend-euclid?lang=cpp>. (閲覧日: 2026-03-31).
4. けんちゃん. AtCoder 版! マスター・オブ・整数 (最大公約数編). <https://qiita.com/drken/items/0c88a37eec520f82b788>. (閲覧日: 2026-03-31).

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)

